

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
KHOA ĐIỆN - ĐIỆN TỬ
BỘ MÔN KỸ THUẬT VIỄN THÔNG

BÁO CÁO ĐỀ TÀI TỐT NGHIỆP

**Đề tài: HỆ THỐNG KIỂM SOÁT PHƯƠNG TIỆN
RA VÀO
ỨNG DỤNG XỬ LÝ ẢNH**

Sinh viên thực hiện : LÊ SÔNG THAO

**Lớp - Khóa : Kỹ thuật Điện tử Tin học Công
nghiệp 2 - K62**

Giảng viên hướng dẫn : TS. Nguyễn Thúy Bình

Hà Nội 2026

Mục lục

TÓM TẮT	5
DANH MỤC TỪ VIẾT TẮT	6
1 TỔNG QUAN ĐỀ TÀI	7
1.1 Đặt vấn đề	7
1.2 Mục tiêu đề tài	8
1.3 Phạm vi thực hiện	8
1.4 Bài toán tổng thể	9
1.5 Sơ đồ khối tổng quan hệ thống	10
1.6 Ý nghĩa thực tiễn của đề tài	11
2 CƠ SỞ LÝ THUYẾT	12
2.1 Tổng quan các khối lý thuyết	12
2.2 Khối nhận diện biên số	12
2.3 Khối điều khiển tự động	17
2.4 Khối phân cứng và giao tiếp	19
2.5 Khối lưu trữ dữ liệu	20
3 THIẾT KẾ VÀ ĐÁNH GIÁ HỆ THỐNG	22
3.1 Môi trường thực nghiệm	22
3.2 Nguyên tắc thiết kế	23
3.3 Kiến trúc phần mềm tổng thể	23
3.4 Luồng dữ liệu trong hệ thống	24
3.5 Thiết kế khối nhận diện biên số	26
3.6 Cấu hình huấn luyện YOLOv8	28

3.7	Kết quả huấn luyện YOLOv8	30
3.7.1	Chỉ số đánh giá chính	30
3.7.2	Biểu đồ quá trình huấn luyện	30
3.7.3	Đường cong Precision, Recall, F1-Score	31
3.7.4	Ma trận nhầm lẫn YOLOv8	32
3.7.5	Phân bố nhãn trong dataset	33
3.7.6	Kết quả validation	34
3.8	Kết quả nhận diện ký tự (PaddleOCR)	34
3.8.1	So sánh phương pháp tiền xử lý ảnh	34
3.8.2	Kết quả hậu xử lý	35
3.8.3	Báo cáo phân lớp theo từng ký tự	35
3.8.4	Phân bố ký tự trong tập đánh giá	37
3.8.5	Ma trận nhầm lẫn OCR	37
3.8.6	Ví dụ nhận diện biển số	38
3.9	Thiết kế luồng xe vào	38
3.10	Thiết kế luồng xe ra	40
3.11	Thiết kế khối cơ sở dữ liệu	41
3.12	Thiết kế khối phần cứng barrier	42
3.13	Đánh giá chức năng điều khiển barrier	45
3.14	Thiết kế đa luồng	46
3.15	Kết quả giao diện và vận hành	47
3.16	Đánh giá ưu điểm	48
3.17	Hạn chế	48
3.18	Hướng phát triển	49
3.19	Kết luận	49

Danh sách hình vẽ

1.1	Sơ đồ khối tổng quan hệ thống	11
2.1	Pipeline nhận diện biển số	16
2.2	Quy trình huấn luyện YOLOv8 phát hiện biển số	17
2.3	Cơ chế an toàn khi đóng barrier	18
2.4	Sơ đồ cơ sở dữ liệu	21
3.1	Sơ đồ module phần mềm	24
3.2	Luồng dữ liệu trong hệ thống	25
3.3	Biểu đồ tổng hợp quá trình huấn luyện YOLOv8 qua 100 epochs	30
3.4	Đường cong Precision và Recall theo epoch	31
3.5	Đường cong F1-Score và PR Curve	31
3.6	Ma trận nhầm lẫn YOLOv8	32
3.7	Phân bố nhãn và kích thước bounding box trong dataset	33
3.8	So sánh Ground Truth và Prediction trên tập validation	34
3.9	Biểu đồ so sánh các phương pháp tiền xử lý ảnh	34
3.10	Biểu đồ so sánh trước và sau hậu xử lý	35
3.11	Phân bố ký tự trong tập đánh giá OCR	37
3.12	Ma trận nhầm lẫn OCR	37
3.13	Ví dụ nhận diện biển số trên tập test	38
3.14	Lưu đồ xử lý xe vào	39
3.15	Lưu đồ xử lý xe ra	41
3.16	Sơ đồ đấu nối phần cứng Arduino – Servo – Cảm biến	45
3.17	Cơ chế đa luồng trong hệ thống	46

Danh sách bảng

2.1	Giao thức lệnh gửi từ máy tính xuống Arduino	20
3.1	Vai trò các bước trong khối nhận diện biển số	28
3.2	Cấu hình huấn luyện YOLOv8	29
3.3	Kết quả huấn luyện YOLOv8 (epoch 100)	30
3.4	So sánh phương pháp tiền xử lý ảnh	34
3.5	So sánh trước và sau hậu xử lý	35
3.6	Báo cáo phân lớp chi tiết OCR theo từng ký tự	36
3.7	Thiết kế các bảng dữ liệu chính	42

TÓM TẮT

Báo cáo trình bày quá trình thiết kế và xây dựng mô hình hệ thống bãi đỗ xe tự động ứng dụng nhận diện biển số. Hệ thống sử dụng hai camera cho cổng vào và cổng ra, mô hình YOLOv8 để phát hiện vùng biển số, PaddleOCR để nhận dạng ký tự, thuật toán hậu xử lý theo định dạng biển số Việt Nam để chọn kết quả cuối, SQLite để quản lý dữ liệu và Arduino để điều khiển hai servo barrier thông qua tín hiệu cảm biến LM393.

Nội dung báo cáo được trình bày theo hướng từ tổng quát đến chi tiết: trước hết xác định bài toán và sơ đồ khối tổng quan, sau đó phân tích cơ sở lý thuyết của từng khối, tiếp theo là thiết kế hệ thống theo luồng dữ liệu và luồng điều khiển, cuối cùng đánh giá kết quả thực nghiệm. Trọng tâm kỹ thuật của đề tài là khối nhận diện biển số và khối điều khiển phần cứng barrier an toàn.

DANH MỤC TỪ VIẾT TẮT

Thuật ngữ	Viết đầy đủ	Nghĩa sử dụng
AI	Artificial Intelligence	Trí tuệ nhân tạo
CLAHE	Contrast Limited Adaptive Histogram Equalization	Cân bằng lược đồ xám thích nghi
COM	Communication Port	Cổng giao tiếp nối tiếp
CPU	Central Processing Unit	Bộ xử lý trung tâm
GPU	Graphics Processing Unit	Bộ xử lý đồ họa
GND	Ground	Chân mass/đất chung
IoU	Intersection over Union	Tỷ lệ giao/hợp giữa hai hộp
mAP	mean Average Precision	Độ chính xác trung bình
OCR	Optical Character Recognition	Nhận dạng ký tự quang học
PWM	Pulse Width Modulation	Điều chế độ rộng xung
SQLite	Structured Query Language Lite	Hệ cơ sở dữ liệu nhúng
USB	Universal Serial Bus	Chuẩn kết nối máy tính
VCC	Voltage Common Collector	Chân cấp nguồn dương
YOLO	You Only Look Once	Mô hình phát hiện đối tượng
YOLOv8	YOLO version 8	Phiên bản 8 của YOLO

Chương 1

TỔNG QUAN ĐỀ TÀI

1.1 Đặt vấn đề

Trong các bãi đỗ xe truyền thống, việc kiểm soát phương tiện thường được thực hiện thủ công thông qua nhân viên bảo vệ, vé giấy hoặc ghi chép bằng tay. Cách làm này có một số hạn chế như tốn nhân lực, dễ nhầm lẫn khi ghi nhận biển số, khó kiểm tra lịch sử vào ra và chưa đảm bảo tính tự động trong quá trình vận hành.

Với sự phát triển của xử lý ảnh, trí tuệ nhân tạo và vi điều khiển, việc xây dựng một hệ thống bãi đỗ xe tự động có khả năng nhận diện biển số, quản lý dữ liệu và điều khiển barrier (thanh chắn/barie) là hướng tiếp cận phù hợp cho các mô hình bãi xe thông minh. Hệ thống có thể tự động ghi nhận biển số phương tiện, kiểm tra điều kiện cho phép vào hoặc ra, đồng thời điều khiển barrier (thanh chắn/barie) thông qua phần cứng.

Đề tài này tập trung xây dựng mô hình bãi đỗ xe tự động sử dụng camera (thiết bị thu hình) để thu nhận hình ảnh biển số, mô hình YOLOv8 (You Only Look Once version 8 - mô hình phát hiện đối tượng phiên bản 8) để phát hiện vùng biển số, PaddleOCR (Paddle Optical Character Recognition - bộ nhận dạng ký tự quang học) để nhận dạng ký tự, SQLite (Structured Query Language Lite - hệ cơ sở dữ liệu nhúng dạng file) để lưu trữ dữ liệu và Arduino (bo mạch vi điều khiển) để điều khiển servo (động cơ điều khiển theo góc quay) barrier (thanh chắn/barie) dựa trên tín hiệu cảm biến vật cản LM393 (module cảm biến vật cản dùng IC so sánh LM393).

1.2 Mục tiêu đề tài

Mục tiêu chính của đề tài là xây dựng một hệ thống mô phỏng bãi đỗ xe tự động có khả năng vận hành theo cả chế độ tự động và thủ công. Hệ thống cần đáp ứng các yêu cầu sau:

- Thu nhận hình ảnh từ camera (thiết bị thu hình) cổng vào và camera (thiết bị thu hình) cổng ra.
- Nhận diện biển số xe từ ảnh chụp camera (thiết bị thu hình).
- Kiểm tra biển số với danh sách biển số hợp lệ.
- Quản lý danh sách xe đang có trong bãi.
- Kiểm tra số lượng chỗ trống trước khi cho xe vào.
- Kiểm tra xe có trong bãi trước khi cho xe ra.
- Điều khiển barrier (thanh chắn/barie) cổng vào và cổng ra bằng servo (động cơ điều khiển theo góc quay).
- Đọc tín hiệu cảm biến để xác định xe đang chắn barrier (thanh chắn/barie) hay đã đi qua.
- Đóng barrier (thanh chắn/barie) sau khi cảm biến không còn phát hiện vật cản trong một khoảng thời gian an toàn.
- Hiển thị trạng thái camera (thiết bị thu hình), biển số, số lượng xe, nhật ký và trạng thái phản cứng trên giao diện.

1.3 Phạm vi thực hiện

Đề tài được thực hiện ở mức mô hình đồ án, tập trung vào việc chứng minh nguyên lý hoạt động của một hệ thống bãi xe tự động. Phạm vi của hệ thống gồm:

- Mô hình bãi xe quy mô nhỏ với số lượng chỗ giới hạn.

- Hai cổng hoạt động độc lập: cổng vào và cổng ra.
- Hai camera (thiết bị thu hình) tương ứng với hai cổng.
- Một mạch Arduino (bo mạch vi điều khiển) điều khiển hai servo (động cơ điều khiển theo góc quay) và đọc hai cảm biến LM393 (module cảm biến vật cản dùng IC so sánh LM393).
- Phần mềm điều khiển chạy trên máy tính bằng Python (ngôn ngữ lập trình Python).
- Cơ sở dữ liệu cục bộ dùng SQLite (hệ cơ sở dữ liệu nhúng dạng file).
- Nhận diện biển số trong điều kiện camera (thiết bị thu hình) được lắp đặt và căn chỉnh phù hợp.

Đề tài chưa mở rộng đến các chức năng thương mại như tính phí tự động, thanh toán điện tử, quản lý người dùng nhiều cấp hoặc triển khai trên hệ thống máy chủ từ xa.

1.4 Bài toán tổng thể

Bài toán của hệ thống có thể mô tả như sau: khi một phương tiện đi tới cổng vào hoặc cổng ra, cảm biến phát hiện vật cản và gửi trạng thái về máy tính thông qua Arduino (bo mạch vi điều khiển). Phần mềm lấy khung hình từ camera (thiết bị thu hình) tương ứng, nhận diện biển số xe, kiểm tra dữ liệu trong cơ sở dữ liệu rồi quyết định có mở barrier (thanh chắn/barie) hay không. Sau khi phương tiện đi qua, cảm biến không còn phát hiện xe, hệ thống chờ thêm một khoảng thời gian an toàn rồi hạ barrier (thanh chắn/barie).

Quy trình này cần đảm bảo ba yêu cầu quan trọng:

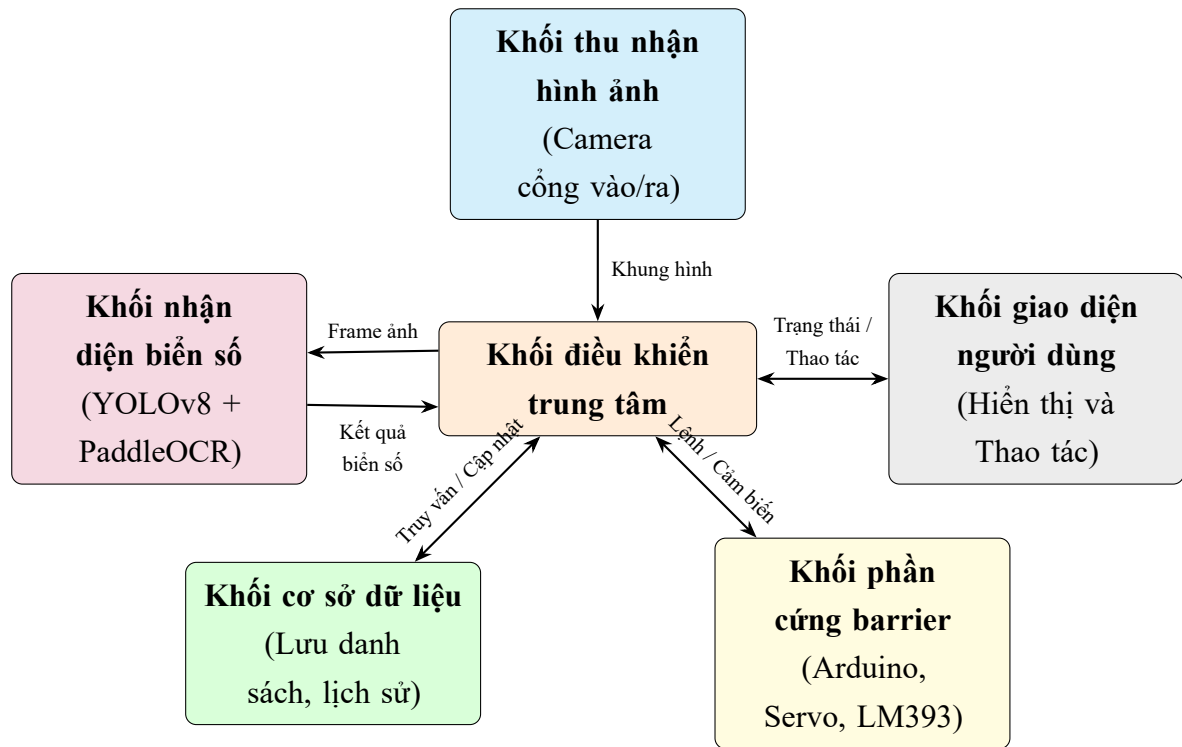
- **Đúng dữ liệu:** biển số phải được nhận diện và đối chiếu chính xác.
- **Đúng logic vận hành:** xe vào, xe ra, bãi đầy, xe không hợp lệ phải được xử lý khác nhau.
- **An toàn cơ khí:** barrier (thanh chắn/barie) không được hạ ngay khi xe vẫn còn nằm trong vùng cảm biến.

1.5 Sơ đồ khối tổng quan hệ thống

Hình 1.1 mô tả cấu trúc tổng quát của hệ thống. Hệ thống được chia thành sáu khối chính:

- **Khối thu nhận hình ảnh:** gồm camera (thiết bị thu hình) cổng vào và camera (thiết bị thu hình) cổng ra, có nhiệm vụ cung cấp khung hình cho hệ thống xử lý.
- **Khối nhận diện biển số:** phát hiện vùng biển số, xử lý ảnh, nhận dạng ký tự và trả về chuỗi biển số.
- **Khối điều khiển trung tâm:** tiếp nhận dữ liệu từ camera (thiết bị thu hình), cảm biến, cơ sở dữ liệu và quyết định điều khiển barrier (thanh chắn/barie).
- **Khối cơ sở dữ liệu:** lưu danh sách biển số hợp lệ, xe trong bãi, sức chứa và lịch sử vào ra.
- **Khối giao diện người dùng:** hiển thị camera (thiết bị thu hình), kết quả nhận diện, trạng thái hệ thống và cho phép thao tác thủ công.
- **Khối phần cứng barrier (thanh chắn/barie):** gồm Arduino (bo mạch vi điều khiển), servo (động cơ điều khiển theo góc quay) và cảm biến LM393 (module cảm biến vật cản dùng IC so sánh LM393) để thực hiện đóng mở cổng.

Hình 1.2 minh họa cách bố trí phần cứng trong mô hình. Mỗi cổng có một camera (thiết bị thu hình) để thu ảnh biển số, một cảm biến LM393 (module cảm biến vật cản dùng IC so sánh LM393) để phát hiện xe tại vùng barrier (thanh chắn/barie) và một servo (động cơ điều khiển theo góc quay) để nâng hạ thanh chắn. Camera (thiết bị thu hình) cần được đặt sao cho biển số nằm trọn trong khung hình, còn cảm biến cần nằm tại vùng xe đi qua để hỗ trợ cơ chế đóng barrier (thanh chắn/barie) an toàn.



Hình 1.1: Sơ đồ khối tổng quan hệ thống

1.6 Ý nghĩa thực tiễn của đề tài

Hệ thống giúp mô phỏng một quy trình quản lý bãi xe tự động có tính thực tế cao. Thay vì chỉ nhận diện biển số đơn lẻ, đề tài kết hợp cả phần mềm, cơ sở dữ liệu, giao diện và phần cứng điều khiển. Điều này giúp người thực hiện hiểu rõ mối liên hệ giữa xử lý ảnh, điều khiển tự động và quản lý dữ liệu trong một hệ thống nhúng kết hợp máy tính.

Chương 2

CƠ SỞ LÝ THUYẾT

2.1 Tổng quan các khối lý thuyết

Hệ thống bãi đỗ xe tự động trong đề tài được xây dựng từ nhiều khối kỹ thuật khác nhau. Các khối này không hoạt động riêng lẻ mà liên kết với nhau thành một chuỗi xử lý thống nhất:

1. Camera thu nhận hình ảnh phương tiện.
2. Mô hình nhận diện tìm vùng biển số.
3. Bộ OCR (Optical Character Recognition - nhận dạng ký tự quang học) đọc ký tự trên biển số.
4. Bộ hậu xử lý chuẩn hóa kết quả theo định dạng biển số Việt Nam.
5. Bộ điều khiển trung tâm kiểm tra dữ liệu và quyết định mở/đóng cổng.
6. Arduino (bo mạch vi điều khiển) thực hiện điều khiển servo (động cơ điều khiển theo góc quay) và gửi trạng thái cảm biến.
7. Cơ sở dữ liệu lưu trữ trạng thái bãi xe.
8. Giao diện hiển thị và cho phép người dùng điều khiển.

2.2 Khối nhận diện biển số

Khối nhận diện biển số là khối kỹ thuật trọng tâm của hệ thống, đóng vai trò quyết định dữ liệu đầu vào cho toàn bộ quá trình quản lý xe. Thay vì chỉ dùng một mô hình OCR đơn giản, hệ thống thiết kế một luồng xử lý (pipeline) phức tạp và chặt chẽ gồm nhiều bước nối tiếp nhau nhằm đảm bảo độ chính xác cao nhất trong nhiều điều kiện môi trường. Quá trình này được thực hiện tuần tự qua 6 bước như sau:

Bước 1: Tiếp nhận nguồn ảnh đầu vào

- **Nguồn ảnh:** Lấy trực tiếp từ luồng video (stream) của camera công vào hoặc công ra.
- **Định dạng:** Mỗi khung hình (frame) được trích xuất là một ma trận điểm ảnh (numpy array) ở hệ màu BGR (định dạng chuẩn của OpenCV).
- **Mục tiêu:** Cung cấp khung hình toàn cảnh chứa phương tiện và biển số cho hệ thống phân tích.

Bước 2: Phát hiện và cắt vùng biển số (YOLOv8)

- **Mô hình và Cấu hình:** Sử dụng mô hình YOLOv8n (bản nano nhẹ và nhanh). Trọng số mô hình `bien_so_yolo.pt` đã được huấn luyện trước trên tập dữ liệu biển số Việt Nam (100 epochs, kích thước ảnh 640x640, batch 16).
- **Thông số dự đoán:** Hàm nhận diện chạy với độ tin cậy tối thiểu `conf=0.25`, kích thước xử lý `imgsz=640` và chạy trên môi trường CPU.
- **Kết quả xử lý:** YOLO trả về tọa độ hộp giới hạn (bounding box - `x1`, `y1`, `x2`, `y2`) của các vùng nghi ngờ là biển số.
- **Cắt ảnh (Crop):** Hệ thống dùng tọa độ này để cắt (crop) riêng vùng biển số ra khỏi ảnh toàn cảnh. Việc cắt nhỏ ảnh giúp OCR chạy nhanh hơn và không bị nhiễu bởi các chữ viết xuất hiện trên phông nền (như chữ trên áo, biển quảng cáo).

Hình 2.1. Chuỗi xử lý (pipeline) nhận diện biển số từ ảnh camera.

Bước 3: Tiền xử lý - Tạo các phiên bản ảnh (OpenCV)

- **Số lượng ảnh sinh ra:** Từ 1 ảnh crop vùng biển số ban đầu, hệ thống tạo ra 3 phiên bản ảnh khác nhau để tối đa hóa khả năng đọc của OCR trong mọi điều kiện ánh sáng.
- **Môi trường:** Xử lý bằng thư viện OpenCV.
- **Các phiên bản:**

1. **Ảnh gốc (BGR):** Giữ lại màu sắc tự nhiên, hoạt động tốt khi điều kiện ánh sáng lý tưởng. 2. **Ảnh CLAHE:** Ảnh được chuyển sang ảnh xám (grayscale) và áp dụng thuật toán CLAHE (Contrast Limited Adaptive Histogram Equalization) với thông số `clipLimit=2.0`, `tileGridSize=(8, 8)`. Phương pháp này giúp cân bằng sáng, làm rõ nét chữ khi biển số bị lóa đèn hoặc nằm trong vùng sấp bóng. 3. **Ảnh Otsu:** Lấy ảnh CLAHE đi qua bộ lọc `cv2.threshold` với cờ `THRESH_BINARY + THRESH_OTSU` để phân ngưỡng tự động. Chữ và nền được tách biệt hoàn toàn thành 2 màu Đen/Trắng, vô cùng hiệu quả với ảnh có độ tương phản cao.

Bước 4: Nhận dạng ký tự quang học (PaddleOCR)

- **Môi trường:** Sử dụng mô hình PaddleOCR mã nguồn mở.
- **Quá trình thực thi:** 3 phiên bản ảnh (Gốc, CLAHE, Otsu) được lần lượt đưa vào PaddleOCR một cách độc lập.
- **Ghép dòng tạo ứng viên:** Với các biển số xe máy gồm 2 dòng, PaddleOCR thường nhận diện thành nhiều mảnh rời rạc. Hệ thống sẽ thu thập toàn bộ các dòng chữ đọc được, ghép nối chúng lại thành một chuỗi văn bản hoàn chỉnh. Tập hợp chuỗi được ghép và các mảnh rời này tạo thành một danh sách các "ứng viên"(candidates) tiềm năng.

Hình 2.3. Chi tiết quá trình sinh 3 phiên bản ảnh, qua OCR và chấm điểm.

Bước 5: Chuẩn hóa và Sửa lỗi theo vị trí (Post-processing) Hậu xử lý là bắt buộc vì mô hình OCR tổng quát không hiểu luật biển số Việt Nam và thường nhầm lẫn giữa số và chữ (VD: 0 và O, 1 và I, 5 và S).

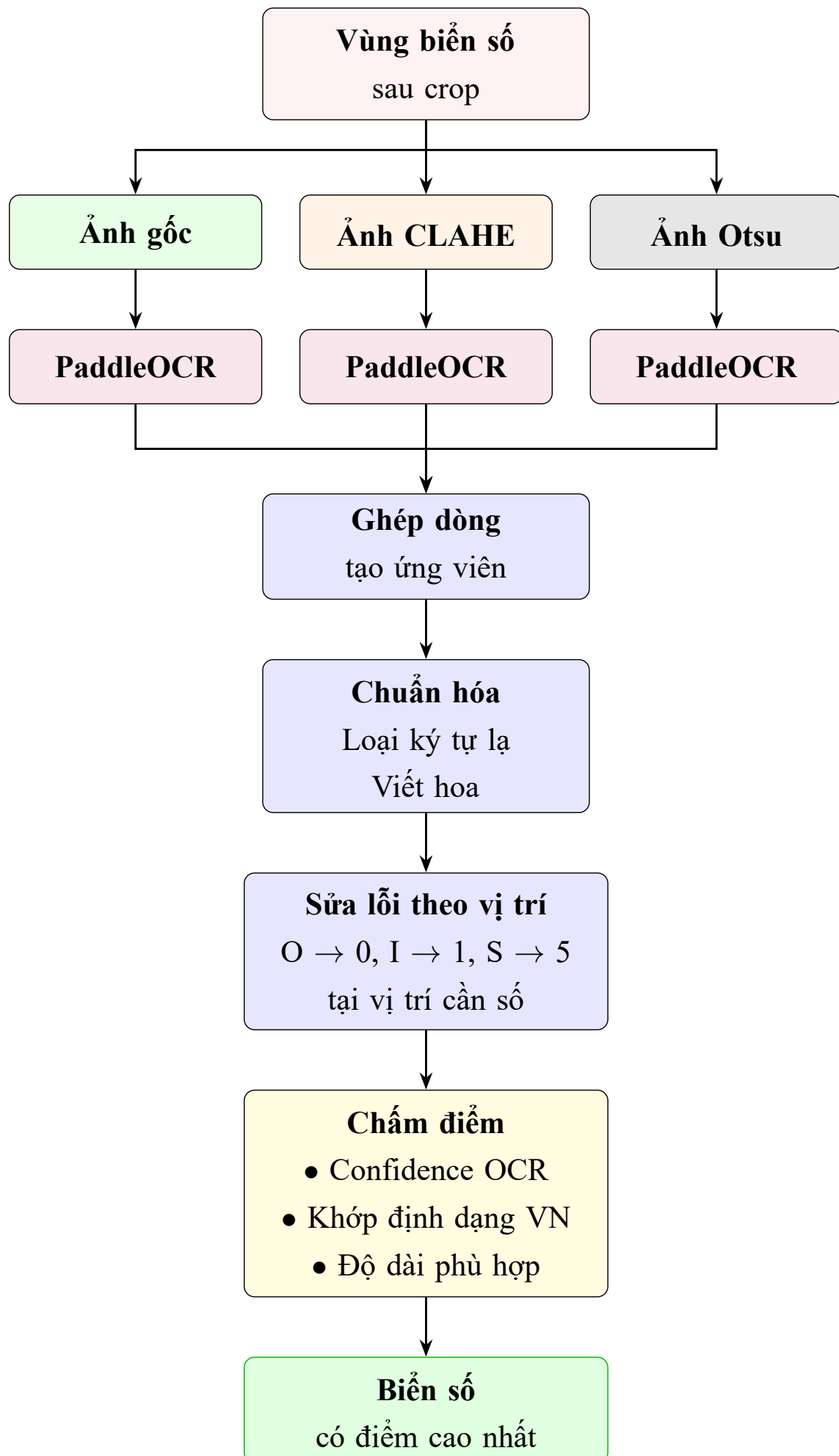
- **Chuẩn hóa:** Dùng Biểu thức chính quy (Regex) `[^A-Za-z0-9]` để loại bỏ các ký tự không phải chữ số/chữ.
- **Sửa lỗi theo vị trí:** Hệ thống rà soát chuỗi ký tự theo luật biển số VN. Ví dụ:
 - 2 ký tự đầu tiên (mã tỉnh) bắt buộc phải là số. Nếu OCR đọc ra chữ O, I, S, Z, hệ thống tự động ép kiểu về 0, 1, 5, 2.
 - Ký tự thứ 3 (seri) bắt buộc là chữ. Nếu OCR ra số 0, 8, hệ thống ép về chữ D, B.

Bước 6: Chấm điểm và Lựa chọn kết quả cuối cùng

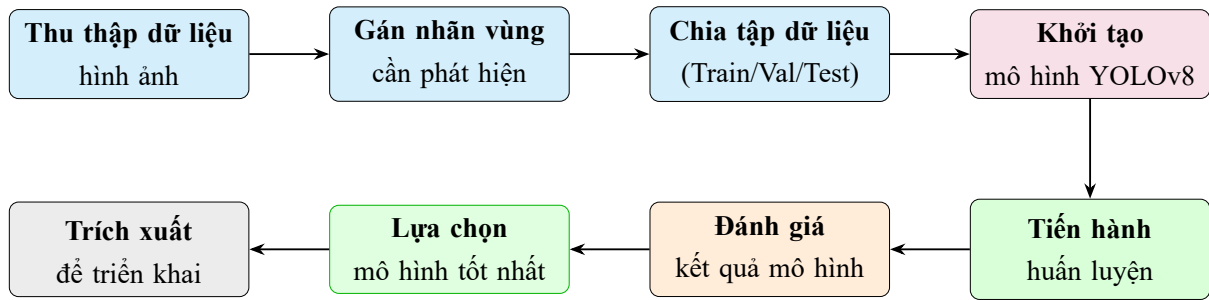
- **Quy tắc chấm điểm:** Mỗi ứng viên (sau khi chuẩn hóa) được chấm điểm dựa trên 3 yếu tố:

1. ***Khớp định dạng (Regex):*** Chấm 1.0 điểm (hoàn hảo) nếu khớp định dạng Ô tô (2 số + 1 chữ + 5 số), Xe máy (2 số + 1 chữ + 6 số) hoặc Biển đặc biệt (2 số + 2 chữ + 5 số). Chấm 0.9 điểm nếu khớp một phần. 2. ***Độ tin cậy OCR:*** Cộng thêm điểm trung bình độ tin cậy (Confidence score) mà PaddleOCR trả về. 3. ***Độ dài phù hợp:*** Cộng điểm thưởng nhỏ (0.01 - 0.04) nếu độ dài chuỗi là 8 hoặc 9 ký tự (chuẩn độ dài biển số VN).

- **Kết quả:** Hệ thống so sánh điểm số của tất cả các ứng viên sinh ra từ cả 3 nhánh ảnh. Ứng viên có tổng điểm cao nhất sẽ được chọn làm kết quả chuỗi biển số đầu ra để ghi vào Cơ sở dữ liệu. Đồng thời, hệ thống tạo ra một ảnh hiển thị phụ (nền đen chữ trắng) của biển số để hiển thị trực quan lên giao diện người dùng.



Hình 2.1: Pipeline nhận diện biển số



Hình 2.2: Quy trình huấn luyện YOLOv8 phát hiện biển số

2.3 Khối điều khiển tự động

Khối điều khiển trung tâm hoạt động dựa trên sự kiện. Các sự kiện chính gồm:

- Cảm biến cổng vào phát hiện xe.
- Cảm biến cổng ra phát hiện xe.
- AI (Artificial Intelligence - trí tuệ nhân tạo) trả về kết quả biển số.
- Người dùng bấm nút điều khiển thủ công.
- Camera bật/tắt hoặc mất khung hình.
- Arduino (bo mạch vi điều khiển) gửi trạng thái cảm biến.

Khi nhận được sự kiện, bộ điều khiển không xử lý độc lập từng sự kiện mà phải xét trạng thái hiện tại của toàn hệ thống. Ví dụ, nếu cảm biến cổng vào phát hiện xe nhưng bãi đã đầy thì không được mở barrier (thanh chắn/barie). Nếu xe ở cổng ra nhưng biển số không tồn tại trong danh sách xe đang ở trong bãi thì cũng không được mở cổng ra.

Điểm quan trọng của khối điều khiển là nó giữ vai trò ra quyết định, không để camera (thiết bị thu hình), AI (trí tuệ nhân tạo) hoặc Arduino (bo mạch vi điều khiển) tự mở cổng. Camera (thiết bị thu hình) chỉ cung cấp ảnh, AI (trí tuệ nhân tạo) chỉ trả về biển số, Arduino (bo mạch vi điều khiển) chỉ thực thi lệnh phân cứng. Quyết định cuối cùng như cho vào, từ chối, cho ra hoặc giữ barrier (thanh chắn/barie) mở đều nằm ở bộ điều khiển trung tâm.

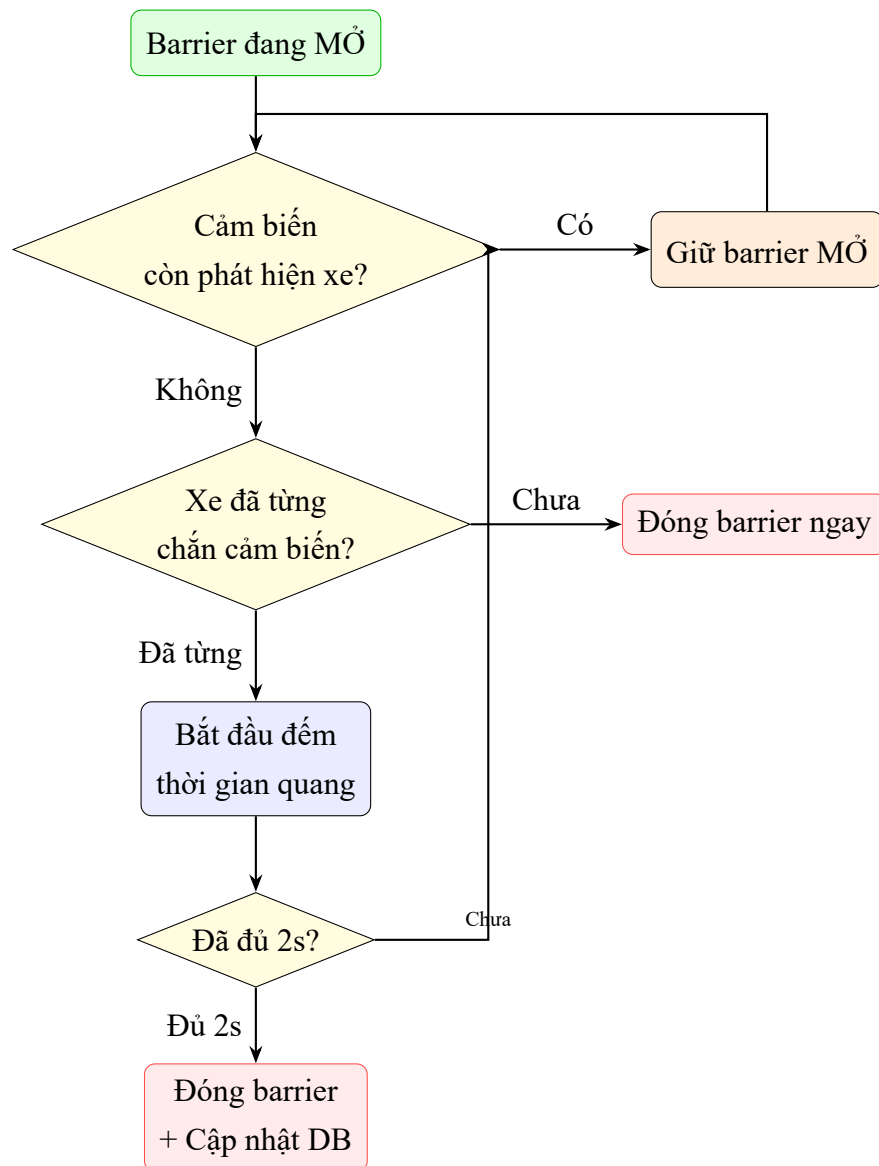
Hình 2.4 mô tả logic đóng barrier (thanh chắn/barie) an toàn. Các mũi tên thể hiện luồng kiểm tra lặp lại: nếu cảm biến còn phát hiện xe thì barrier (thanh

chắn/barie) tiếp tục giữ mở; nếu cảm biến đã quang và duy trì đủ 2 giây thì hệ thống mới gửi lệnh đóng.

Barrier (thanh chắn/barie) không được đóng ngay khi có lệnh đóng nếu cảm biến vẫn đang phát hiện vật cản. Cơ chế an toàn được thiết kế như sau:

1. Khi barrier (thanh chắn/barie) đang mở, hệ thống theo dõi cảm biến tương ứng. 2. Nếu cảm biến còn phát hiện xe, hệ thống giữ barrier (thanh chắn/barie) ở trạng thái mở. 3. Khi cảm biến không còn phát hiện xe, hệ thống bắt đầu đếm thời gian. 4. Sau 2 giây không có vật cản, hệ thống mới gửi lệnh đóng barrier (thanh chắn/barie).

Cơ chế này giúp mô hình vận hành thực tế hơn, tránh trường hợp barrier (thanh chắn/barie) hạ quá sớm khi xe chưa đi qua hoàn toàn.



Hình 2.3: Cơ chế an toàn khi đóng barrier

2.4 Khối phần cứng và giao tiếp

Arduino (bo mạch vi điều khiển) đóng vai trò là bộ điều khiển phần cứng. Trong đề tài, Arduino nhận lệnh từ máy tính qua Serial (giao tiếp nối tiếp), điều khiển hai servo (động cơ điều khiển theo góc quay) và đọc trạng thái hai cảm biến LM393 (module cảm biến vật cản dùng IC so sánh LM393).

Arduino (bo mạch vi điều khiển) giúp tách phần điều khiển cơ khí khỏi phần mềm máy tính. Máy tính chỉ cần gửi lệnh mức cao như mở cổng vào hoặc đóng cổng ra, còn Arduino thực hiện điều khiển servo (động cơ điều khiển theo góc quay) ở mức tín hiệu.

Servo (động cơ điều khiển theo góc quay) là động cơ có khả năng quay đến một góc xác định. Trong mô hình, servo được dùng để nâng và hạ thanh barrier (thanh chắn/barie). Hệ thống quy ước:

- Góc đóng: barrier (thanh chắn/barie) nằm ngang, không cho xe qua.
- Góc mở: barrier (thanh chắn/barie) nâng lên, cho xe đi qua.

Để chuyển động thực tế hơn, servo (động cơ điều khiển theo góc quay) không quay tức thời từ góc đóng sang góc mở, mà được điều khiển từng bước nhỏ với độ trễ giữa các bước.

Cảm biến LM393 (module cảm biến vật cản dùng IC so sánh LM393) được dùng để phát hiện vật cản tại cổng vào và cổng ra. Khi có xe chắn vùng cảm biến, tín hiệu đầu ra thay đổi. Arduino (bo mạch vi điều khiển) đọc tín hiệu này và gửi trạng thái về máy tính.

Trong hệ thống, cảm biến không chỉ dùng để phát hiện xe xuất hiện mà còn dùng để quyết định thời điểm an toàn để đóng barrier (thanh chắn/barie).

Hình 2.5 mô tả giao thức truyền nhận giữa máy tính và Arduino (bo mạch vi điều khiển). Mũi tên từ máy tính sang Arduino là luồng lệnh điều khiển; mũi tên từ Arduino về máy tính là luồng phản hồi trạng thái cảm biến và thông tin kiểm tra kết nối.

Máy tính và Arduino (bo mạch vi điều khiển) giao tiếp thông qua cổng Serial (giao tiếp nối tiếp). Giao thức được thiết kế đơn giản bằng chuỗi văn bản, giúp dễ kiểm tra bằng Serial Monitor (cửa sổ giám sát giao tiếp nối tiếp) và dễ mở rộng.

Một số lệnh từ máy tính gửi xuống Arduino (bo mạch vi điều khiển) được trình bày trong Bảng 2.1.

Bảng 2.1. Giao thức lệnh gửi từ máy tính xuống Arduino (bo mạch vi điều khiển)

Dữ liệu Arduino (bo mạch vi điều khiển) gửi lên máy tính có dạng:

“text CAM_BIEN:xe_vao=1,xe_ra=0 ‘

Trong đó

Trong đó xe_{vao} và xe_{ra}

Kênh Serial (giao tiếp nối tiếp) là hai chiều. Chiều từ máy tính xuống Arduino (bo mạch vi điều khiển) là lệnh điều khiển như mở hoặc đóng barrier (thanh chắn/barie). Chiều từ Arduino lên máy tính là trạng thái cảm biến và thông tin phản hồi. Việc dùng chuỗi văn bản làm giao thức giúp quá trình kiểm tra khi ghép nối phần cứng trở nên đơn giản: có thể mở Serial Monitor (cửa sổ giám sát giao tiếp nối tiếp) để gửi thử $SERVO_{TEST}$, $PING$ hoặc quan sát dòng

Bảng 2.1: Giao thức lệnh gửi từ máy tính xuống Arduino

Lệnh	Ý nghĩa
VAO_MO	Mở barrier cổng vào
VAO_DONG	Đóng barrier cổng vào
RA_MO	Mở barrier cổng ra
RA_DONG	Đóng barrier cổng ra
PING	Kiểm tra phản hồi Arduino
SERVO_TEST	Kiểm tra hoạt động servo

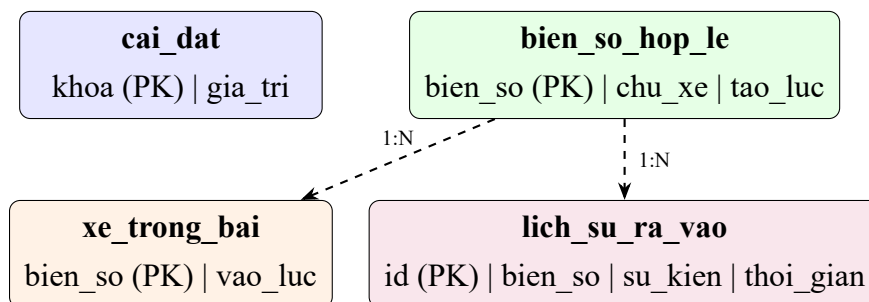
2.5 Khôi lưu trữ dữ liệu

Hình 2.6 mô tả các nhóm dữ liệu chính trong hệ thống. Bảng xe_trong_b thể hiện trạng thái hiện tại của bãi xe, còn bảng $lich_su_ra_vao$ lưu lại các sự kiện đã xảy ra để phục vụ theo dõi.

SQLite (Structured Query Language Lite - hệ cơ sở dữ liệu nhúng dạng file) được sử dụng vì phù hợp với mô hình cục bộ, không cần

cài đặt máy chủ cơ sở dữ liệu riêng. Cơ sở dữ liệu gồm bốn nhóm dữ liệu chính:

- **Cấu hình:** lưu sức chứa bãi xe.
- **Biển số hợp lệ:** lưu danh sách biển số được phép vào.
- **Xe trong bãi:** lưu các xe đang hiện diện trong bãi.
- **Lịch sử vào ra:** lưu nhật ký sự kiện phục vụ theo dõi và kiểm tra.



Hình 2.4: Sơ đồ cơ sở dữ liệu

Chương 3

THIẾT KẾ VÀ ĐÁNH GIÁ HỆ THỐNG

3.1 Môi trường thực nghiệm

Hệ thống được thử nghiệm trên mô hình bãi xe quy mô nhỏ. Môi trường thực nghiệm gồm:

- Máy tính chạy phần mềm Python (ngôn ngữ lập trình Python) với giao diện Tkinter (thư viện giao diện đồ họa của Python).
- Camera USB/Full HD (camera dùng cổng USB, độ phân giải Full HD) dùng cho cổng vào và cổng ra.
- Arduino (bo mạch vi điều khiển) kết nối với máy tính qua cổng USB (Universal Serial Bus - chuẩn kết nối giữa máy tính và thiết bị).
- Hai servo (động cơ điều khiển theo góc quay) dùng để đóng/mở barrier (thanh chắn/barie).
- Hai cảm biến LM393 (module cảm biến vật cản dùng IC so sánh LM393) dùng để phát hiện xe tại cổng vào và cổng ra.
- Cơ sở dữ liệu SQLite (hệ cơ sở dữ liệu nhúng dạng file) lưu cục bộ trên máy tính.

Camera được căn chỉnh sao cho biển số nằm trong vùng quan sát, hạn chế bị cắt mất mép và giảm ảnh hưởng của ánh sáng chói.

3.2 Nguyên tắc thiết kế

Hệ thống được thiết kế theo hướng chia thành các khối chức năng độc lập. Mỗi khối đảm nhiệm một vai trò cụ thể, giúp quá trình kiểm tra, bảo trì và mở rộng dễ dàng hơn. Cách chia khối cũng giúp báo cáo và mô hình hóa hệ thống rõ ràng hơn.

Các nguyên tắc thiết kế chính:

- Tách biệt giao diện, xử lý logic, nhận diện biển số, lưu trữ dữ liệu và điều khiển phần cứng.
- Luồng camera (thiết bị thu hình) và luồng AI (Artificial Intelligence - trí tuệ nhân tạo) không làm treo giao diện.
- Mọi quyết định mở cổng đều phải đi qua bộ điều khiển trung tâm.
- Cơ sở dữ liệu là nguồn xác định trạng thái xe trong bãi.
- Barrier chỉ được đóng khi cảm biến xác nhận vùng cổng đã an toàn.

3.3 Kiến trúc phần mềm tổng thể

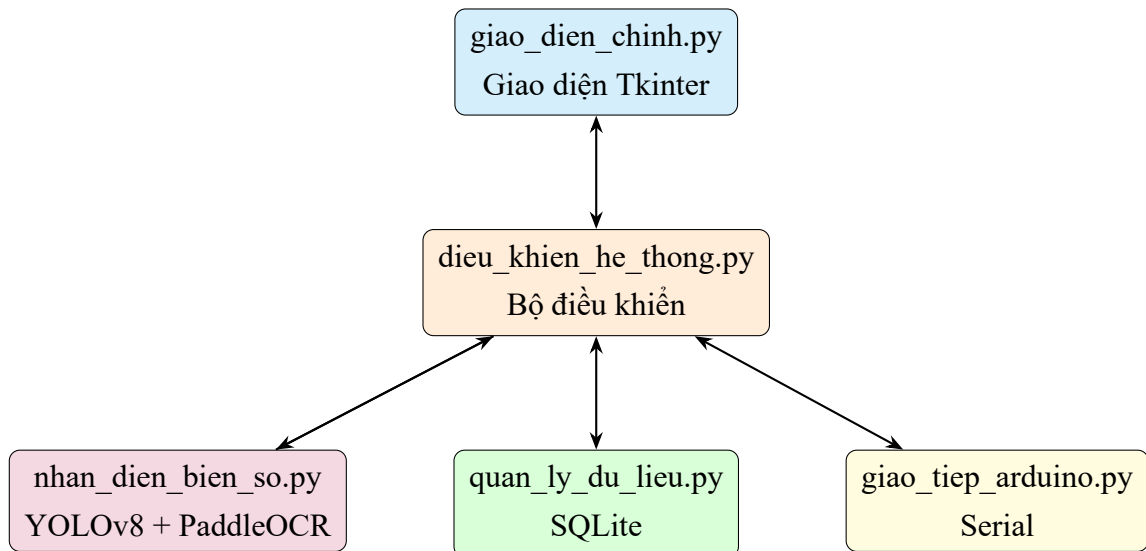
Hình 3.1 cho thấy bộ điều khiển trung tâm là điểm phối hợp chính giữa giao diện, camera (thiết bị thu hình), AI (trí tuệ nhân tạo), cơ sở dữ liệu và Arduino (bo mạch vi điều khiển). Các module (khối chức năng phần mềm) không tự quyết định mở cổng mà gửi dữ liệu về bộ điều khiển để xử lý thống nhất.

Hệ thống phần mềm gồm các khối sau:

- Khối giao diện: hiển thị thông tin và nhận thao tác người dùng.

- Khối điều khiển trung tâm: điều phối toàn bộ hoạt động của hệ thống.
- Khối camera (thiết bị thu hình): đọc hình ảnh từ camera (thiết bị thu hình) vào và camera ra.
- Khối AI (Artificial Intelligence - trí tuệ nhân tạo) nhận diện: xử lý ảnh và trả về biến số.
- Khối cơ sở dữ liệu: lưu trữ và truy vấn thông tin xe.
- Khối giao tiếp Arduino (bo mạch vi điều khiển): gửi lệnh và đọc trạng thái phần cứng.

Khối điều khiển trung tâm là nơi liên kết các khối còn lại. Camera (thiết bị thu hình), AI (trí tuệ nhân tạo), cơ sở dữ liệu và Arduino (bo mạch vi điều khiển) không tự quyết định mở barrier (thanh chắn/ba mọi quyết định đều được đưa về khối điều khiển trung tâm để đảm bảo thống nhất logic.



Hình 3.1: Sơ đồ module phần mềm

3.4 Luồng dữ liệu trong hệ thống

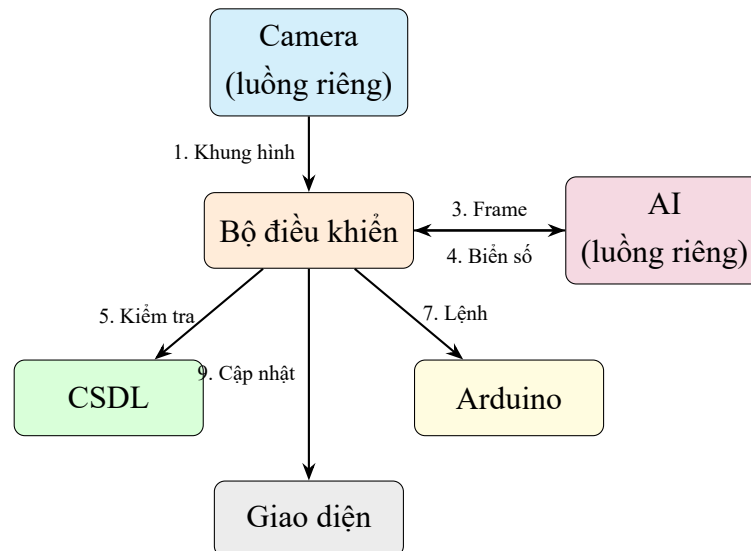
Hình 3.2 mô tả hướng di chuyển của dữ liệu giữa các khối. Dữ liệu ảnh đi từ camera (thiết bị thu hình) đến bộ điều khiển và AI (trí

tuệ nhân tạo); dữ liệu biến số đi ngược về bộ điều khiển để kiểm tra cơ sở dữ liệu; lệnh điều khiển đi từ bộ điều khiển xuống Arduino (bo mạch vi điều khiển).

Luồng dữ liệu chính của hệ thống gồm:

1. Camera gửi khung hình mới nhất cho bộ điều khiển.
2. Khi có yêu cầu nhận diện, bộ điều khiển lấy khung hình và gửi vào hàng đợi AI (trí tuệ nhân tạo).
3. Khối AI (trí tuệ nhân tạo) trả về biến số và ảnh biến số đã xử lý để hiển thị.
4. Bộ điều khiển kiểm tra biến số với cơ sở dữ liệu.
5. Nếu đủ điều kiện, bộ điều khiển gửi lệnh mở barrier (thanh chắn/barie) cho Arduino (bo mạch vi điều khiển).
6. Arduino (bo mạch vi điều khiển) điều khiển servo (động cơ điều khiển theo góc quay) và gửi trạng thái cảm biến về máy tính.
7. Giao diện cập nhật trạng thái, nhật ký và kết quả nhận diện.

Việc tổ chức luồng dữ liệu theo hướng này giúp hạn chế xung đột giữa các tác vụ có tốc độ khác nhau. Camera (thiết bị thu hình) cần cập nhật liên tục, AI (trí tuệ nhân tạo) xử lý chậm hơn, còn giao diện cần phản hồi ổn định cho người dùng.



Hình 3.2: Luồng dữ liệu trong hệ thống

3.5 Thiết kế khối nhận diện biển số

Khối nhận diện biển số trong hệ thống được thiết kế như một dịch vụ xử lý ảnh độc lập. Đầu vào của khối là một frame (khung hình) BGR (Blue - Green - Red - thứ tự kênh màu xanh dương, xanh lá, đỏ trong OpenCV) lấy từ camera (thiết bị thu hình) cổng vào hoặc cổng ra. Đầu ra gồm hai phần: chuỗi biển số đã chuẩn hóa để bộ điều khiển kiểm tra logic, và ảnh biển số đã xử lý để hiển thị cho người dùng quan sát. Cách tách đầu ra như vậy giúp hệ thống vừa có dữ liệu dạng text (văn bản) để xử lý tự động, vừa có hình ảnh minh họa để người vận hành biết AI (trí tuệ nhân tạo) đang đọc từ vùng nào.

Khi bắt đầu chương trình, mô hình YOLO (You Only Look Once - mô hình phát hiện đối tượng một giai đoạn) và PaddleOCR (bộ nhận dạng ký tự quang học) được nạp vào bộ nhớ. Việc nạp mô hình thường tốn thời gian ở lần đầu, nên hệ thống có bước warm-up (chạy khởi động) để tránh trường hợp xe đầu tiên bị xử lý chậm bất thường. Khi cảm biến phát hiện xe, bộ điều khiển không gửi toàn bộ luồng video sang AI (trí tuệ nhân tạo), mà chỉ lấy frame (khung hình) mới nhất tại thời điểm cần nhận diện. Cách làm này giảm tải cho CPU (Central Processing Unit - bộ xử lý trung tâm)/GPU (Graphics Processing Unit - bộ xử lý đồ họa) và phù hợp với bài toán bãi xe: chỉ cần nhận diện khi có xe ở cổng, không cần OCR (nhận dạng ký tự quang học) liên tục 30 khung hình mỗi giây.

Ở bước phát hiện vùng biển, YOLO (mô hình phát hiện đối tượng một giai đoạn) trả về danh sách hộp giới hạn kèm độ tin cậy. Hệ thống chọn vùng biển số có điểm tin cậy phù hợp rồi crop (cắt vùng ảnh) từ ảnh gốc. Nếu crop quá sát, OCR (nhận dạng ký tự quang học) có thể mất một phần ký tự ở mép; nếu crop quá rộng, OCR dễ đọc nhầm từ nền. Vì vậy trong mô hình thực tế, camera (thiết bị thu hình) cần được đặt sao cho biển số nằm đủ trong khung và kích thước biển số không quá lớn so với ảnh. Đây là lý do phần lắp đặt camera có ảnh hưởng trực tiếp đến độ chính xác nhận diện.

Sau khi có ảnh vùng biển số, hệ thống tạo ba phiên bản ảnh cho OCR (nhận dạng ký tự quang học). Ảnh gốc được giữ lại vì trong thực nghiệm đây là phiên bản cho độ chính xác rất cao khi ánh sáng ổn định. Ảnh CLAHE (cân bằng lược đồ xám thích nghi có giới hạn tương phản) được tạo bằng cách cân bằng tương phản cục bộ, giúp làm rõ nét chữ trong vùng sáng tối không đều. Ảnh Otsu (ảnh nhị phân theo ngưỡng Otsu) được tạo bằng cách nhị phân hóa, có tác dụng tách chữ và nền trong trường hợp biển số có độ tương phản mạnh. Ba ảnh này không nhằm làm đẹp giao diện, mà nhằm tạo nhiều điều kiện đọc khác nhau cho OCR.

Kết quả PaddleOCR (bộ nhận dạng ký tự quang học) có thể gồm nhiều dòng, đặc biệt với biển số xe máy hai dòng. Hệ thống tạo nhiều ứng viên từ kết quả OCR (nhận dạng ký tự quang học): chuỗi ghép tất cả các dòng và từng dòng riêng lẻ. Mỗi ứng viên được chuẩn hóa bằng cách loại bỏ ký tự không phải chữ/số, viết hoa và sửa lỗi theo vị trí. Sau đó hệ thống chấm điểm ứng viên theo ba yếu tố: độ tin cậy trung bình do OCR trả về, mức độ khớp với định dạng biển số Việt Nam và độ dài chuỗi phù hợp. Ứng viên có điểm cao nhất được chọn làm kết quả cuối.

Vai trò của từng bước trong khối nhận diện được tóm tắt trong Bảng 3.1.

Bảng 3.1. Vai trò các bước trong khối nhận diện biển số

Ảnh nền đen/chữ trắng được tạo sau cùng để hiển thị trực quan trên giao diện. Ảnh này giúp người dùng thấy rõ vùng ký tự mà hệ thống đang xử lý, nhưng không phải là dữ liệu duy nhất quyết định kết quả OCR (nhận dạng ký tự quang học). Quyết định nhận diện dựa trên nhiều ứng viên từ ảnh gốc, CLAHE (cân bằng lược đồ xám thích nghi có giới hạn tương phản) và Otsu (phương pháp tự động chọn ngưỡng nhị phân ảnh).

Bảng 3.1: Vai trò các bước trong khối nhận diện biển số

Bước		Mục đích	Lý do cần thiết
YOLOv8	phát hiện	Tìm vùng biển số	Giảm nhiễu nền trước OCR
Crop vùng biển số		Tách vùng quan tâm	Tăng tốc và tăng độ chính xác
Tạo ảnh gốc/CLAHE/Otsu	ảnh	Nhiều điều kiện đọc	Ánh sáng thay đổi
PaddleOCR		Đọc ký tự từ ảnh	Không cần train riêng
Chuẩn hóa chuỗi		Thống nhất text	Loại ký tự lạ
Sửa lỗi theo vị trí		Sửa nhầm chữ/số	Cấu trúc biển số VN
Chấm điểm		Chọn ứng viên tốt	Confidence + luật định dạng

3.6 Cấu hình huấn luyện YOLOv8

Mô hình YOLOv8n được huấn luyện với các siêu tham số sau:

Bảng 3.2: Cấu hình huấn luyện YOLOv8

Tham số	Giá trị	Ý nghĩa
Model	YOLOv8n	Biến thể nano, nhỏ gọn
Pretrained	yolov8n.pt	Trọng số khởi tạo sẵn
Epochs	100	Số vòng huấn luyện
Image size	640	Kích thước ảnh đầu vào
Batch size	16	Số ảnh mỗi lô
Optimizer	auto	Tự chọn tối ưu
lr0	0.01	Learning rate khởi đầu
lrf	0.01	Learning rate cuối
Momentum	0.937	Hệ số quán tính SGD
Weight decay	0.0005	Hệ số phạt trọng số
Warmup epochs	3.0	Số epoch khởi động
Mosaic	1.0	Tăng cường ghép ảnh
Fliplr	0.5	Lật ngang 50%
Scale	0.5	Thay đổi tỷ lệ
Close mosaic	10	Tắt mosaic 10 epoch cuối
AMP	true	Mixed precision training
Device	GPU (0)	Huấn luyện trên GPU

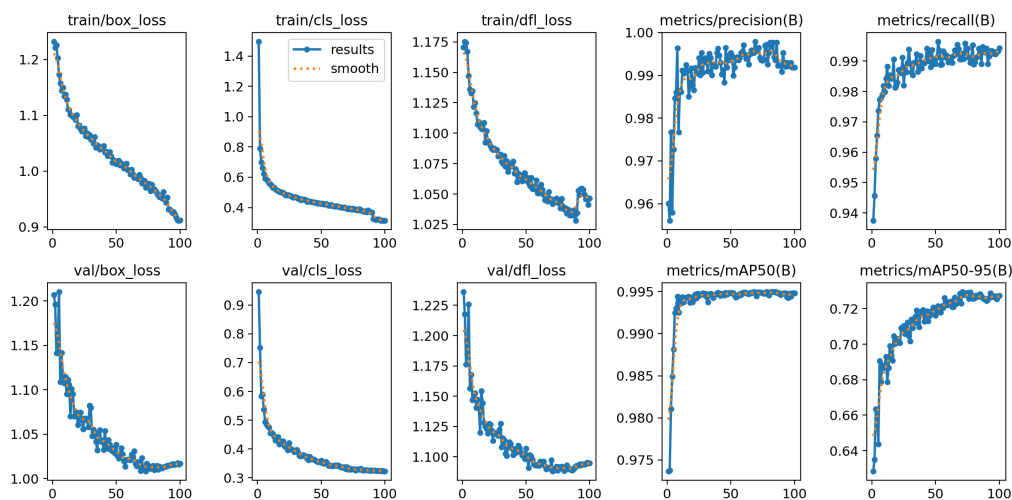
3.7 Kết quả huấn luyện YOLOv8

3.7.1 Chỉ số đánh giá chính

Bảng 3.3: Kết quả huấn luyện YOLOv8 (epoch 100)

Chỉ số	Kết quả
Precision	99.18%
Recall	99.43%
mAP50	99.48%
mAP50-95	72.74%
Train Box Loss (cuối)	0.912
Train Cls Loss (cuối)	0.313
Val Box Loss (cuối)	1.017
Val Cls Loss (cuối)	0.323

3.7.2 Biểu đồ quá trình huấn luyện



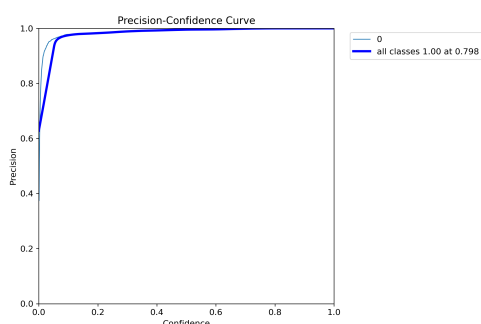
Hình 3.3: Biểu đồ tổng hợp quá trình huấn luyện YOLOv8 qua 100 epochs

Nhận xét:

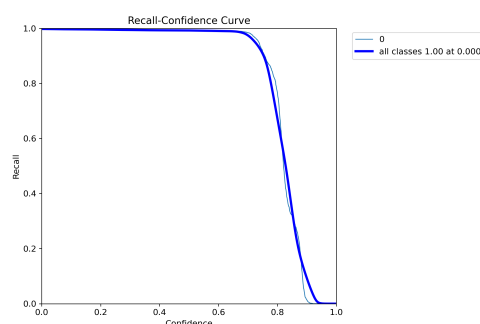
- Train loss giảm đều đặn qua 100 epochs, mô hình học tốt và không bị overfitting.

- Validation loss ổn định từ epoch 50 trở đi, mô hình đã hội tụ.
- Precision và Recall đều đạt trên 99% từ rất sớm.
- mAP50 hội tụ nhanh đạt $\sim 99.48\%$.

3.7.3 Đường cong Precision, Recall, F1-Score

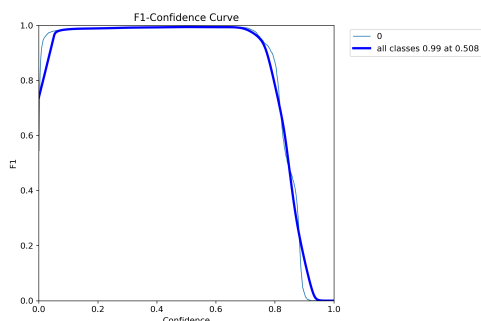


(a) Đường cong Box Precision

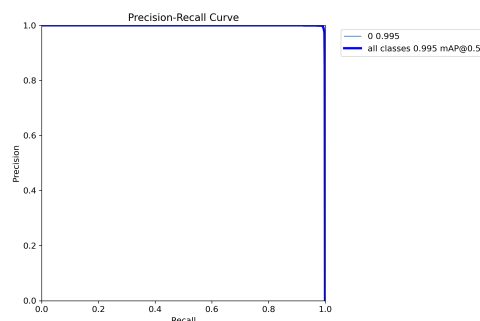


(b) Đường cong Box Recall

Hình 3.4: Đường cong Precision và Recall theo epoch



(a) Đường cong F1-Score

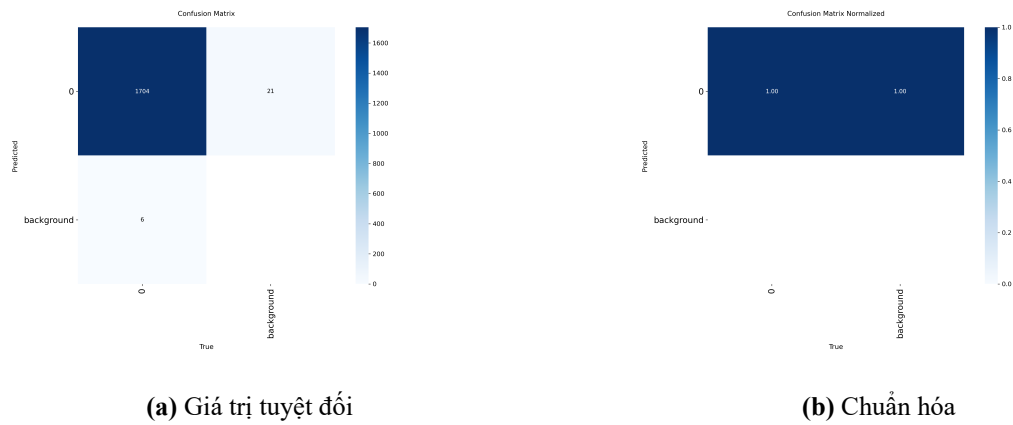


(b) Đường cong Precision-Recall

Hình 3.5: Đường cong F1-Score và PR Curve

F1-Score đạt đỉnh ở ngưỡng confidence phù hợp. Đường PR gần như vuông góc ở góc trên bên phải, xác nhận mAP50 rất cao ($\sim 99.48\%$).

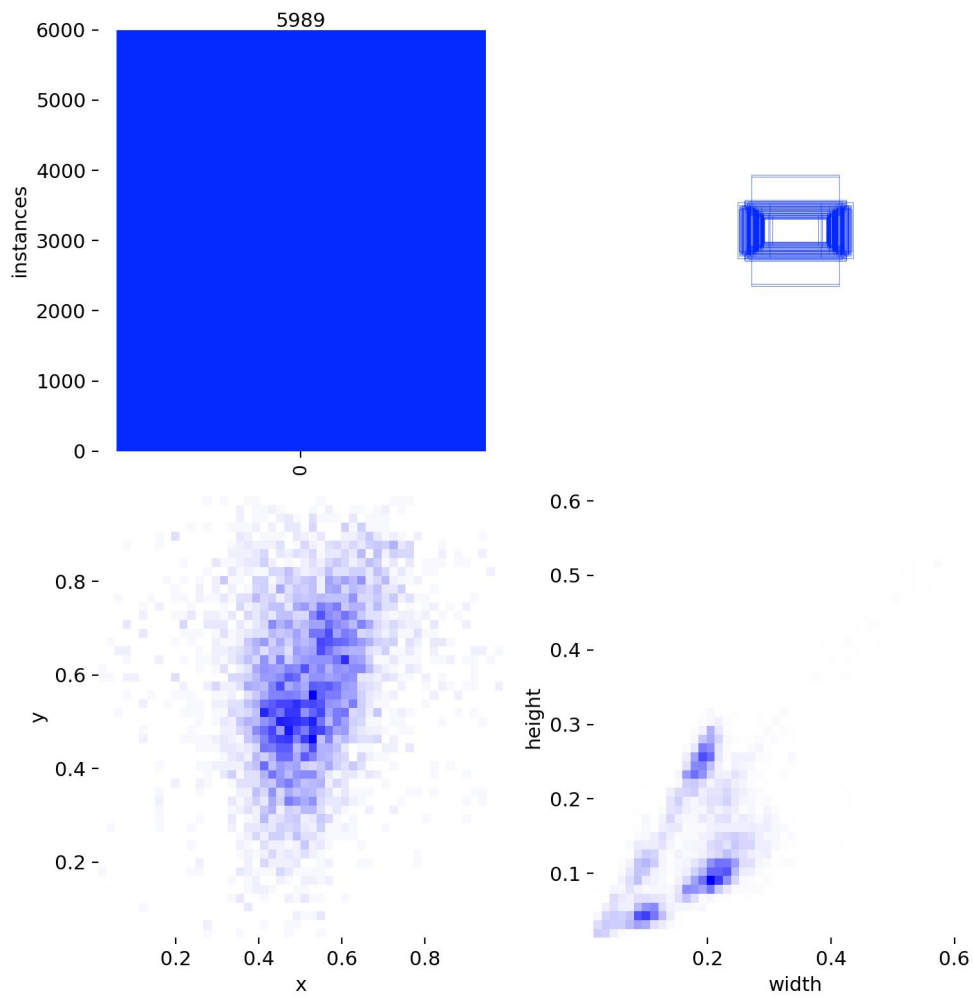
3.7.4 Ma trận nhầm lẫn YOLOv8



Hình 3.6: Ma trận nhầm lẫn YOLOv8

Ma trận nhầm lẫn cho thấy mô hình YOLO hầu như không nhầm lẫn. Tỷ lệ dự đoán đúng lớp license_plate gần 100%, với rất ít false negative hoặc false positive.

3.7.5 Phân bố nhãn trong dataset



Hình 3.7: Phân bố nhãn và kích thước bounding box trong dataset

3.7.6 Kết quả validation



(a) Nhận thực tế (Ground Truth)

(b) Kết quả dự đoán

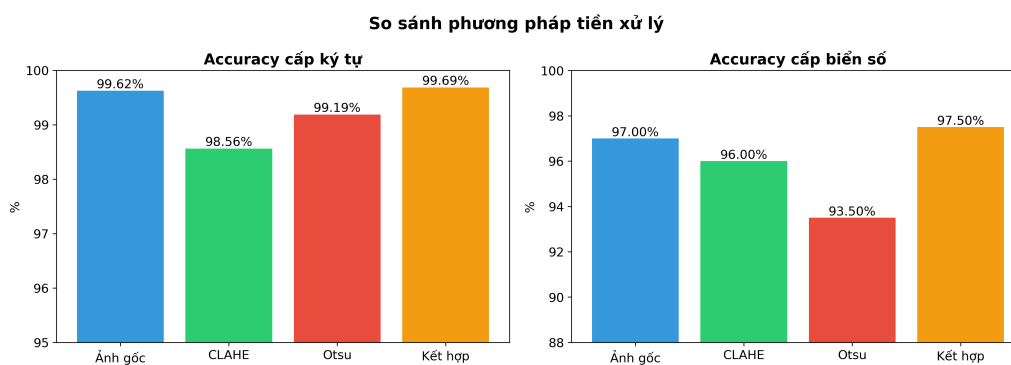
Hình 3.8: So sánh Ground Truth và Prediction trên tập validation

3.8 Kết quả nhận diện ký tự (PaddleOCR)

3.8.1 So sánh phương pháp tiền xử lý ảnh

Bảng 3.4: So sánh phương pháp tiền xử lý ảnh

Phương pháp	Accuracy ký tự	Accuracy biến số
Ảnh gốc	99.62%	97.00%
CLAHE	98.56%	96.00%
Otsu	99.19%	93.50%
Kết hợp chọn tốt nhất	99.69%	97.50%



Hình 3.9: Biểu đồ so sánh các phương pháp tiền xử lý ảnh

3.8.2 Kết quả hậu xử lý

Hậu xử lý theo định dạng biển số Việt Nam giúp sửa một số lỗi nhận dạng ký tự và chọn kết quả hợp lý hơn. Kết quả so sánh trước và sau hậu xử lý được trình bày trong Bảng 4.3.

Bảng 4.3. So sánh trước và sau hậu xử lý

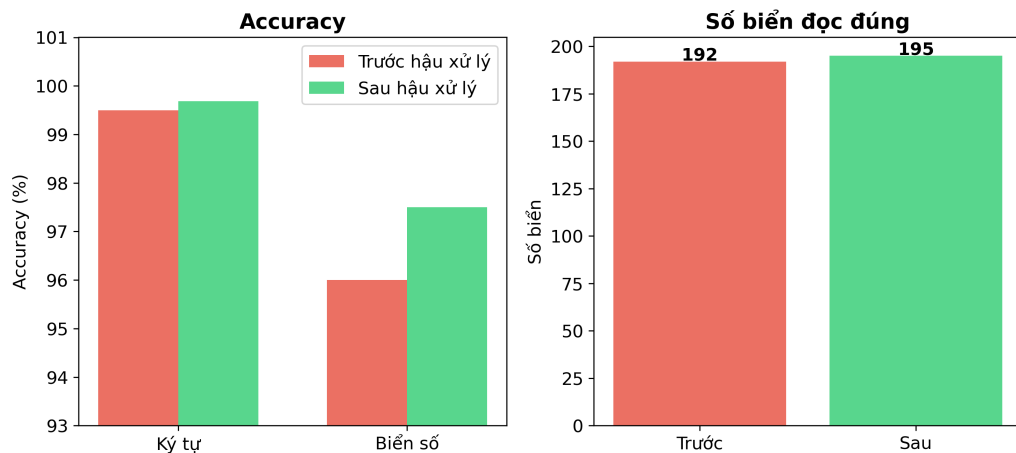
Sau hậu xử lý, độ chính xác cấp biển số tăng từ 95.00

Hình 4.2 cho thấy hậu xử lý theo quy tắc biển số Việt Nam giúp tăng độ chính xác, đặc biệt ở cấp biển số hoàn chỉnh. Điều này chứng minh rằng việc kết hợp OCR (nhận dạng ký tự quang học) với luật định dạng thực tế là cần thiết.

Bảng 3.5: So sánh trước và sau hậu xử lý

Chỉ số	Trước	Sau	Cải thiện
Accuracy cấp ký tự	99.44%	99.69%	+0.25%
Accuracy cấp biển số	95.00%	97.50%	+2.50%

Hiệu quả hậu xử lý theo quy tắc biển số Việt Nam



Hình 3.10: Biểu đồ so sánh trước và sau hậu xử lý

3.8.3 Báo cáo phân lớp theo từng ký tự

Đánh giá chi tiết khả năng nhận diện từng ký tự trên tập test 200 ảnh (tổng 1600 ký tự):

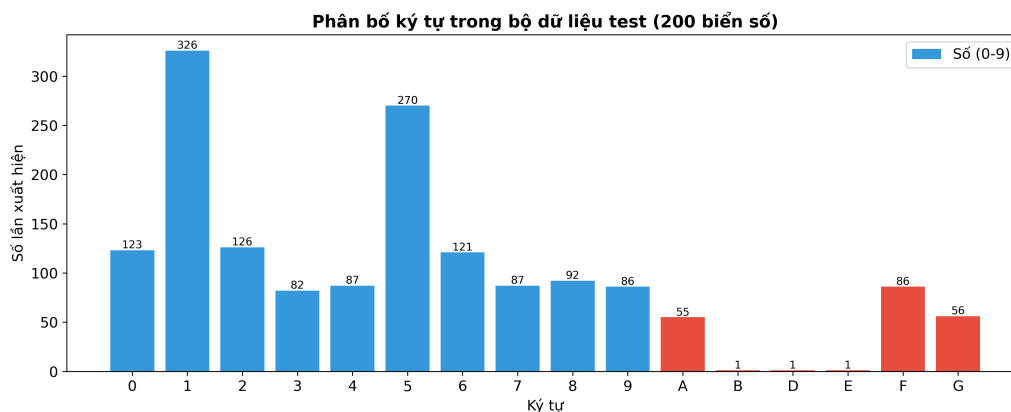
Bảng 3.6: Báo cáo phân lớp chi tiết OCR theo từng ký tự

Ký tự	Precision	Recall	F1-Score	Số mẫu
0	1.00	1.00	1.00	123
1	1.00	1.00	1.00	326
2	0.98	0.99	0.99	126
3	1.00	0.99	0.99	82
4	1.00	1.00	1.00	87
5	1.00	1.00	1.00	270
6	0.99	0.99	0.99	121
7	1.00	1.00	1.00	87
8	1.00	1.00	1.00	92
9	0.99	1.00	0.99	86
A	1.00	1.00	1.00	55
B	1.00	1.00	1.00	1
D	1.00	1.00	1.00	1
E	1.00	1.00	1.00	1
F	1.00	1.00	1.00	86
G	1.00	1.00	1.00	56
Weighted avg	1.00	1.00	1.00	1600

Nhận xét:

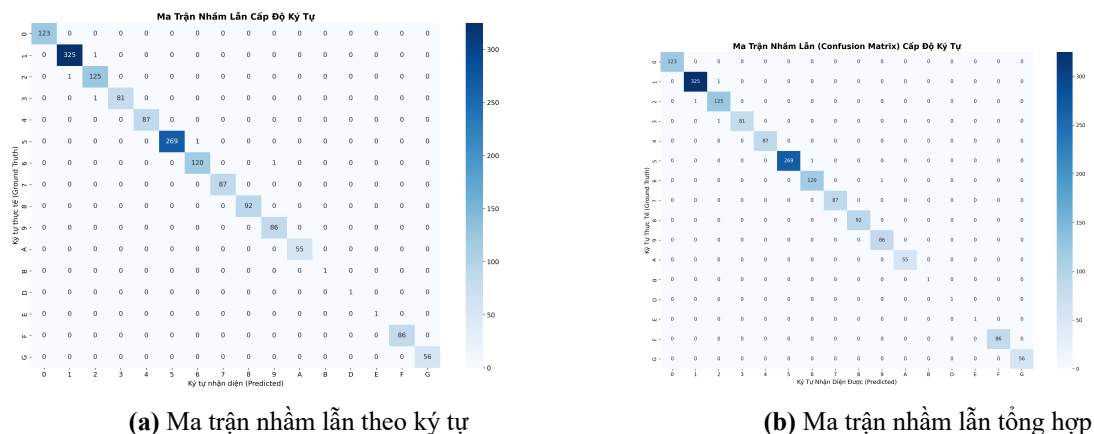
- Accuracy tổng $\sim 100\%$ trên tập 1600 ký tự (sau hậu xử lý).
- Ký tự dễ nhầm nhất: 2 (precision 0.98), 6 và 9 (precision 0.99).
- Ký tự 1 và 5 có số mẫu lớn nhất (326 và 270).

3.8.4 Phân bố ký tự trong tập đánh giá



Hình 3.11: Phân bố ký tự trong tập đánh giá OCR

3.8.5 Ma trận nhầm lẫn OCR



Hình 3.12: Ma trận nhầm lẫn OCR

Ma trận nhầm lẫn OCR cho thấy đường chéo chính rất đậm (hầu hết ký tự được nhận diện đúng). Một số trường hợp nhầm nhẹ giữa các cặp: $2 \leftrightarrow Z$, $6 \leftrightarrow G$, $0 \leftrightarrow O$, $1 \leftrightarrow I$, $5 \leftrightarrow S$ – đây là nhầm lẫn cổ điển mà bước hậu xử lý đã sửa được phần lớn.

3.8.6 Ví dụ nhận diện biển số

Ví dụ nhận diện đúng (trên) và sai (dưới)



Hình 3.13: Ví dụ nhận diện biển số trên tập test

3.9 Thiết kế luồng xe vào

Hình 3.3 mô tả luồng xử lý xe vào theo hướng từ trên xuống. Các nhánh rẽ thể hiện trường hợp từ chối mở cổng khi không đọc được biển số, biển số không hợp lệ, xe đã có trong bãi hoặc bãi đã đầy.

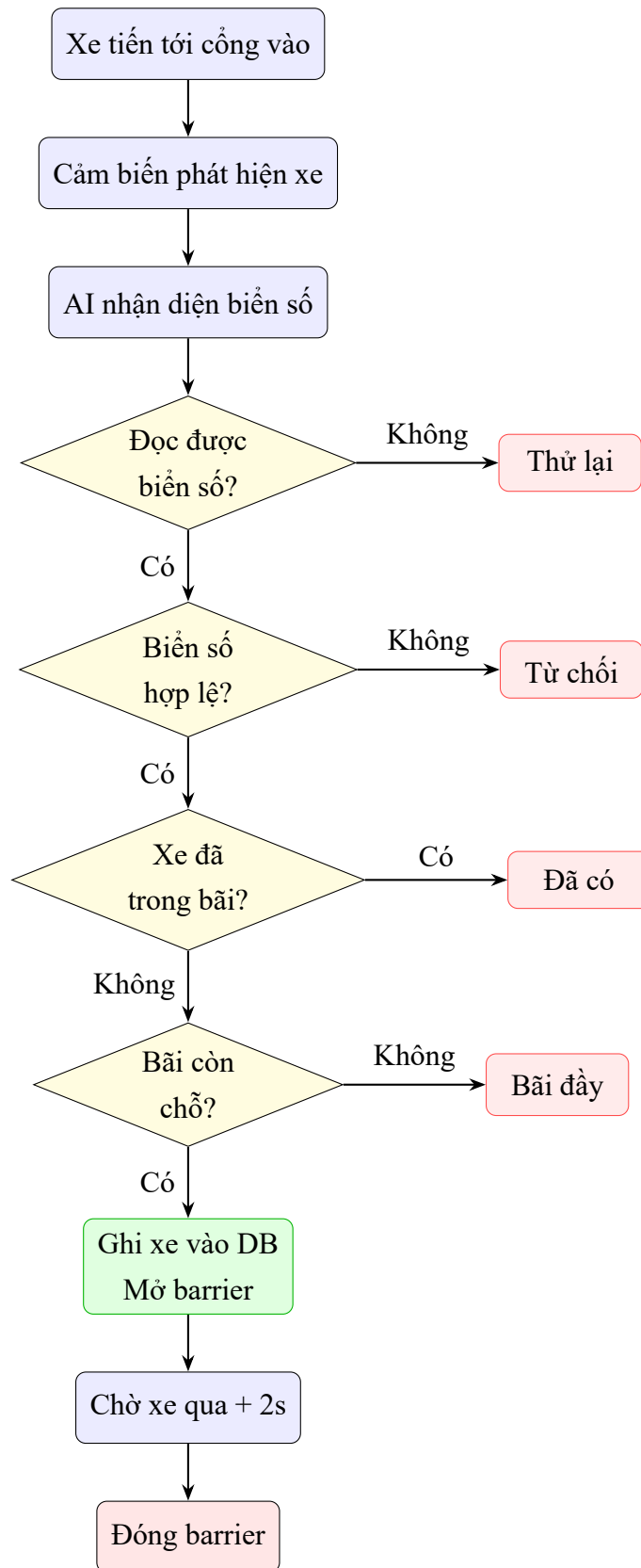
Luồng xe vào được thiết kế như sau:

1. Xe tiến tới cổng vào.
2. Cảm biến cổng vào phát hiện vật cản.
3. Arduino (bo mạch vi điều khiển) gửi trạng thái cảm biến về máy tính.
4. Bộ điều khiển lấy ảnh mới nhất từ camera (thiết bị thu hình) vào.
5. Khối AI (trí tuệ nhân tạo) nhận diện biển số.
6. Bộ điều khiển kiểm tra các điều kiện:

- Biển số có đọc được hay không.
- Biển số có nằm trong danh sách hợp lệ hay không.
- Xe đã có trong bãi hay chưa.
- Bãi còn chỗ trống hay không.

7. Nếu hợp lệ, hệ thống ghi xe vào cơ sở dữ liệu.
8. Bộ điều khiển gửi lệnh mở barrier (thanh chắn/barie) vào.
9. Khi xe đi qua và cảm biến không còn phát hiện vật cản, hệ thống chờ 2 giây.
10. Bộ điều khiển gửi lệnh đóng barrier (thanh chắn/barie) vào.

Nếu một trong các điều kiện kiểm tra không đạt, hệ thống không mở barrier (thanh chắn/barie) và ghi thông báo vào nhật ký.



Hình 3.14: Lưu đồ xử lý xe vào

3.10 Thiết kế luồng xe ra

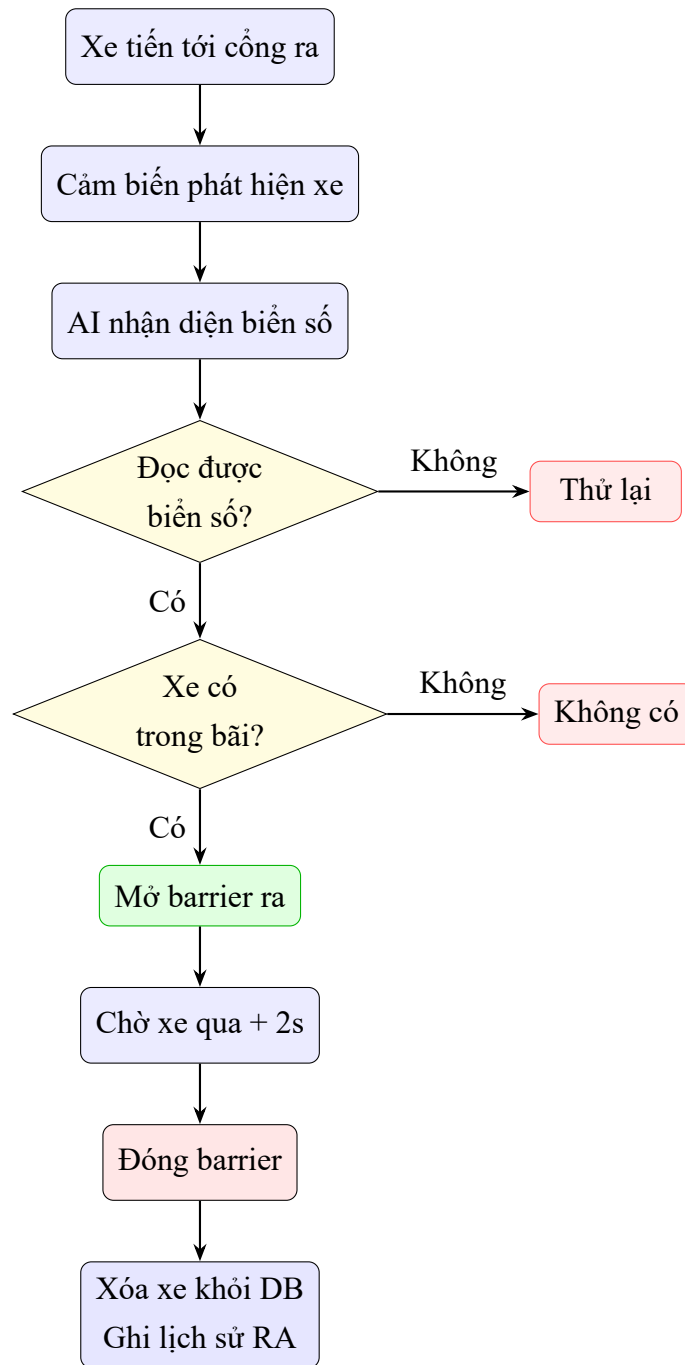
Hình 3.4 mô tả luồng xử lý xe ra. Điểm khác biệt quan trọng là dữ liệu xe chỉ được xóa khỏi bảng xe trong bãi sau khi xe đã đi qua, cảm biến quang và barrier (thanh chắn/barie) đã đóng lại.

Luồng xe ra được thiết kế theo hướng chặt chẽ hơn so với việc chỉ giảm số lượng xe tự động. Hệ thống cần xác định biến số xe ra và kiểm tra xe đó có đang ở trong bãi hay không.

Các bước xử lý:

1. Xe tiến tới cổng ra.
2. Cảm biến cổng ra phát hiện vật cản.
3. Arduino (bo mạch vi điều khiển) gửi trạng thái cảm biến về máy tính.
4. Bộ điều khiển lấy ảnh từ camera (thiết bị thu hình) ra.
5. Khối AI (trí tuệ nhân tạo) nhận diện biến số xe ra.
6. Bộ điều khiển kiểm tra biến số có tồn tại trong danh sách xe đang ở trong bãi không.
7. Nếu có, hệ thống mở barrier (thanh chắn/barie) ra.
8. Xe đi qua cổng ra.
9. Khi cảm biến không còn phát hiện vật cản, hệ thống chờ 2 giây.
10. Bộ điều khiển đóng barrier (thanh chắn/barie) ra.
11. Sau khi đóng barrier (thanh chắn/barie), hệ thống mới xóa biến số khỏi danh sách xe trong bãi và ghi lịch sử ra.

Cách thiết kế này tránh lỗi xóa xe khỏi bãi quá sớm. Nếu barrier (thanh chắn/barie) mở nhưng xe chưa đi qua, dữ liệu vẫn chưa bị cập nhật sai.



Hình 3.15: Lưu đồ xử lý xe ra

3.11 Thiết kế khối cơ sở dữ liệu

Cơ sở dữ liệu gồm bốn bảng chính, được mô tả trong Bảng 3.2.

Bảng 3.2. Thiết kế các bảng dữ liệu chính

Các thao tác chính với cơ sở dữ liệu:

-- Thêm biển số hợp lệ.

- Xóa biến số hợp lệ.
- Kiểm tra biến số có hợp lệ không.
- Thêm xe vào danh sách xe trong bãi.
- Kiểm tra xe có đang trong bãi không.
- Xóa xe khỏi bãi khi xe ra.
- Đếm số xe hiện tại.
- Lấy lịch sử vào ra.

Trong hệ thống, dữ liệu xe trong bãi có vai trò quyết định. Xe chỉ được ra khi biến số được nhận diện tại cổng ra tồn tại trong bảng xe đang ở trong bãi.

Bảng 3.7: Thiết kế các bảng dữ liệu chính

Bảng	Chức năng
cai_dat	Lưu cấu hình hệ thống (sức chứa bãi xe)
bien_so_hop_le	Lưu danh sách biến số được phép vào
xe_trong_bai	Lưu các xe đang có trong bãi
lich_su_ra_vao	Lưu lịch sử xe vào và xe ra

3.12 Thiết kế khối phần cứng barrier

Khối phần cứng gồm:

- Arduino (bo mạch vi điều khiển).
- Hai servo (động cơ điều khiển theo góc quay): một cho cổng vào, một cho cổng ra.
- Hai cảm biến LM393 (module cảm biến vật cản dùng IC so sánh LM393): một ở cổng vào, một ở cổng ra.

- Nguồn cấp cho Arduino (bo mạch vi điều khiển) và servo (động cơ điều khiển theo góc quay).
- Kết nối USB (Universal Serial Bus - chuẩn kết nối giữa máy tính và thiết bị) giữa Arduino (bo mạch vi điều khiển) và máy tính.

Hình 3.5 mô tả sơ đồ đấu nối phần cứng. Các mũi tên từ Arduino (bo mạch vi điều khiển) đến servo (động cơ điều khiển theo góc quay) là tín hiệu điều khiển PWM (Pulse Width Modulation - điều chế độ rộng xung) một chiều: Arduino phát xung điều khiển góc quay cho servo. Các mũi tên từ cảm biến LM393 (module cảm biến vật cản dùng IC so sánh LM393) về Arduino là tín hiệu số một chiều: cảm biến xuất mức logic tại chân DO (Digital Output - ngõ ra số), Arduino đọc tại các chân digital (chân số). Mũi tên giữa máy tính và Arduino là giao tiếp Serial (giao tiếp nối tiếp) hai chiều qua USB (chuẩn kết nối giữa máy tính và thiết bị): máy tính gửi lệnh, Arduino gửi trạng thái cảm biến và phản hồi.

Trong mô hình, servo (động cơ điều khiển theo góc quay) cổng vào nối chân tín hiệu với D9, servo cổng ra nối chân tín hiệu với D10. Cảm biến LM393 (module cảm biến vật cản dùng IC so sánh LM393) cổng vào nối DO (Digital Output - ngõ ra số) với D2, cảm biến LM393 cổng ra nối DO với D3. Hai cảm biến được cấp VCC (Voltage Common Collector - chân cấp nguồn dương) và GND (Ground - chân mass/đất chung) từ Arduino (bo mạch vi điều khiển). Nếu servo dùng nguồn 5V ngoài để tránh sụt áp, bắt buộc GND của nguồn servo phải nối chung với GND Arduino; nếu không, tín hiệu điều khiển từ Arduino sang servo sẽ không có cùng mức điện áp và servo có thể rung hoặc không hoạt động đúng.

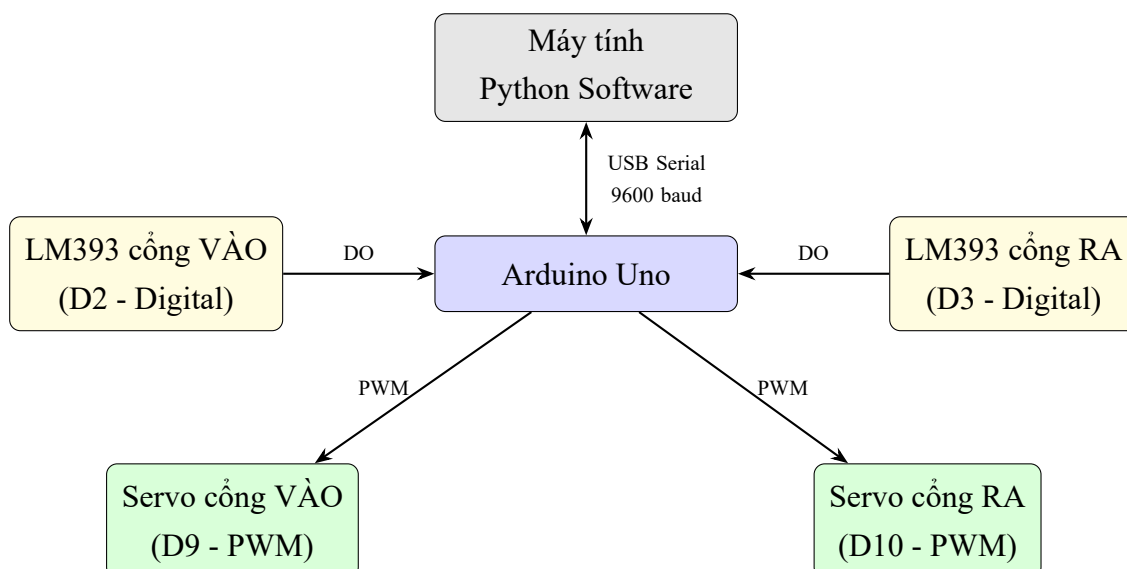
Nguyên lý hoạt động của phần cứng:

1. Arduino (bo mạch vi điều khiển) đọc trạng thái cảm biến theo chu kỳ.
2. Khi trạng thái cảm biến thay đổi hoặc đến chu kỳ gửi trạng thái, Arduino (bo mạch vi điều khiển) gửi dữ liệu lên máy tính.
3. Khi nhận lệnh mở hoặc đóng, Arduino điều khiển servo (động

cơ điều khiển theo góc quay) quay đến góc tương ứng. 4. Servo (động cơ điều khiển theo góc quay) được điều khiển từng bước nhỏ để chuyển động mượt hơn.

Một số lưu ý khi lắp đặt phần cứng:

- Cảm biến LM393 (module cảm biến vật cản dùng IC so sánh LM393) cần được chỉnh biến trở để khoảng cách phát hiện phù hợp với kích thước mô hình.
- Vùng phát hiện của cảm biến nên đặt đúng vị trí xe đi qua barrier (thanh chắn/barie), không đặt quá xa khiến barrier đóng sai thời điểm.
- Servo SG90 (động cơ servo cỡ nhỏ SG90) có dòng khởi động tương đối lớn, nếu dùng nguồn USB (chuẩn kết nối giữa máy tính và thiết bị) yếu có thể làm Arduino (bo mạch vi điều khiển) reset hoặc servo giật.
- Dây tín hiệu servo nên cắm đúng chân điều khiển, dây đỏ là VCC (chân cấp nguồn dương), dây nâu/đen là GND (mass/đất chung), dây cam/vàng là tín hiệu.
- Khi thử phần cứng, nên dùng lệnh SERVO_TEST trước để kiểm tra hai servo (động cơ điều khiển theo góc quay) độc lập với giao diện.
- Barrier (thanh chắn/barie) cơ khí cần nhẹ và không kẹt trục để servo (động cơ điều khiển theo góc quay) không quá tải.



Hình 3.16: Sơ đồ đầu nối phần cứng Arduino – Servo – Cảm biến

3.13 Đánh giá chức năng điều khiển barrier

Khối điều khiển barrier (thanh chắn/barie) hoạt động theo đúng nguyên tắc:

- Khi có lệnh mở, servo (động cơ điều khiển theo góc quay) nâng barrier (thanh chắn/barie) lên.
- Khi xe đi qua, cảm biến chuyển từ trạng thái có vật cản sang không có vật cản.
- Hệ thống chờ 2 giây sau khi cảm biến không còn phát hiện vật cản.
- Sau thời gian chờ, Arduino (bo mạch vi điều khiển) nhận lệnh đóng barrier (thanh chắn/barie).

Việc điều khiển servo (động cơ điều khiển theo góc quay) theo từng bước nhỏ giúp chuyển động barrier (thanh chắn/barie) mềm hơn so với việc quay ngay lập tức đến góc đích. Cơ chế chờ sau khi cảm biến hết vật cản giúp mô hình vận hành sát thực tế hơn.

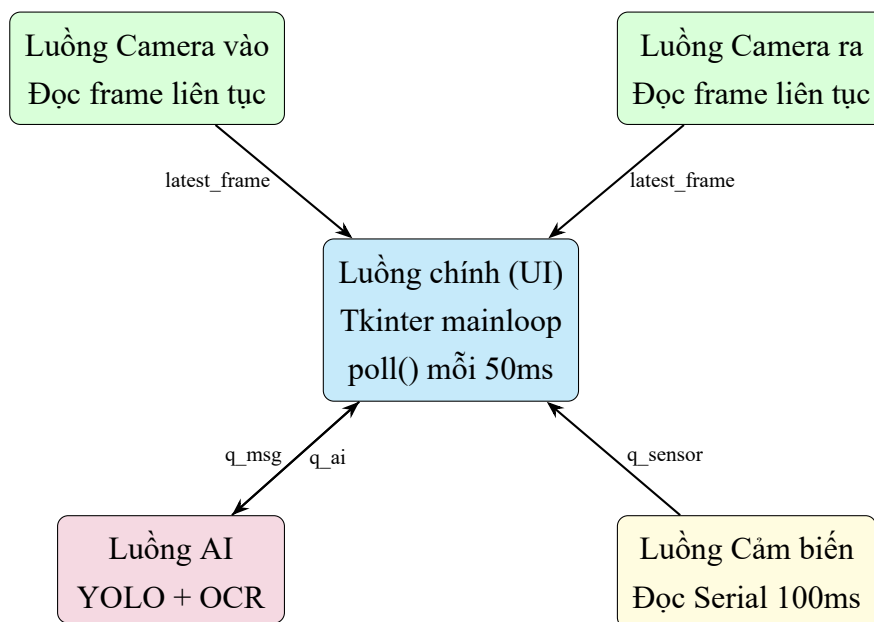
3.14 Thiết kế đa luồng

Hình 3.6 mô tả cách tách các tác vụ có thời gian xử lý khác nhau ra nhiều luồng. Camera (thiết bị thu hình), AI (trí tuệ nhân tạo) và cảm biến chạy tách khỏi luồng giao diện để giao diện không bị treo khi nhận diện hoặc đọc thiết bị.

Hệ thống có nhiều tác vụ diễn ra đồng thời:

- Đọc camera (thiết bị thu hình) liên tục.
- Chạy AI (trí tuệ nhân tạo) nhận diện biển số.
- Đọc trạng thái cảm biến từ Arduino (bo mạch vi điều khiển).
- Cập nhật giao diện.
- Truy vấn cơ sở dữ liệu.

Nếu tất cả tác vụ chạy trên cùng một luồng, giao diện có thể bị treo khi AI (trí tuệ nhân tạo) xử lý lâu hoặc khi camera (thiết bị thu hình) chậm phản hồi. Vì vậy hệ thống sử dụng cách tổ chức đa luồng: camera, AI và cảm biến được xử lý tách khỏi luồng giao diện. Giao diện chỉ lấy trạng thái mới nhất để hiển thị định kỳ.



Hình 3.17: Cơ chế đa luồng trong hệ thống

3.15 Kết quả giao diện và vận hành

Giao diện hệ thống hiển thị được các thành phần chính:

- Hình ảnh camera (thiết bị thu hình) cổng vào.
- Hình ảnh camera (thiết bị thu hình) cổng ra.
- Ảnh biển số sau xử lý của từng cổng.
- Chuỗi biển số nhận diện được.
- Số lượng xe hiện tại và sức chứa bãi.
- Chế độ thủ công/tự động.
- Trạng thái kết nối Arduino (bo mạch vi điều khiển).
- Danh sách biển số hợp lệ.
- Nhật ký sự kiện.

Trong quá trình thử nghiệm, hệ thống thực hiện được các chức năng:

- Bật và tắt từng camera (thiết bị thu hình).
- Chuyển camera (thiết bị thu hình) cho từng cổng.
- Nhận diện biển số ở cổng vào.
- Nhận diện biển số ở cổng ra.
- Thêm và xóa biển số hợp lệ.
- Tự động mở barrier (thanh chắn/barie) khi xe đủ điều kiện.
- Không mở barrier (thanh chắn/barie) khi biển số không hợp lệ.
- Không cho xe vào khi bãi đã đầy.
- Không cho xe ra nếu xe không có trong bãi.
- Đóng barrier (thanh chắn/barie) sau khi cảm biến không còn phát hiện vật cản trong 2 giây.

3.16 Đánh giá ưu điểm

Hệ thống đạt được các ưu điểm chính:

- Có đủ hai luồng cổng vào và cổng ra.
- Mỗi cổng có camera (thiết bị thu hình) và vùng hiển thị biển số riêng.
- Có kiểm tra biển số hợp lệ trước khi cho xe vào.
- Có kiểm tra xe trong bãi trước khi cho xe ra.
- Không xóa xe khỏi bãi trước khi xe thật sự ra khỏi vùng cổng.
- Có chế độ tự động và thủ công.
- Có giao diện trực quan, dễ quan sát trạng thái.
- Cơ sở dữ liệu được tổ chức rõ ràng.
- Giao tiếp Arduino (bo mạch vi điều khiển) đơn giản, dễ kiểm tra và phân tích lỗi khi ghép nối phần cứng.
- Cơ chế đóng barrier (thanh chắn/barie) có xét trạng thái cảm biến, tăng tính an toàn.

3.17 Hạn chế

Bên cạnh các kết quả đạt được, hệ thống vẫn còn một số hạn chế:

- Độ chính xác nhận diện phụ thuộc vào góc đặt camera (thiết bị thu hình) và ánh sáng môi trường.
- Nếu biển số bị đưa quá sát camera (thiết bị thu hình) hoặc bị cắt mất mép, kết quả nhận diện có thể sai.
- Chưa có chức năng lưu ảnh xe vào/ra thành hồ sơ.
- Chưa có chức năng tính tiền gửi xe.

- Chưa có tài khoản quản trị và phân quyền người dùng.
- Mô hình mới dừng ở quy mô đồ án, chưa đánh giá trong môi trường bãi xe thực tế ngoài trời.

3.18 Hướng phát triển

Trong tương lai, hệ thống có thể được phát triển theo các hướng sau:

- Bổ sung chức năng tính phí theo thời gian gửi xe.
- Lưu ảnh xe vào và ảnh xe ra để đối chiếu.
- Thêm chức năng xuất báo cáo lịch sử vào/ra.
- Xây dựng giao diện web hoặc ứng dụng quản lý từ xa.
- Tối ưu nhận diện trong điều kiện thiếu sáng, ngược sáng hoặc biển số bị nghiêng.
- Bổ sung cơ chế cảnh báo khi mất kết nối Arduino (bo mạch vi điều khiển) hoặc camera (thiết bị thu hình).
- Cải tiến mô hình phần cứng để barrier (thanh chắn/barie) chắc chắn và ổn định hơn.

3.19 Kết luận

Đề tài đã xây dựng được mô hình hệ thống bãi đỗ xe tự động có sự kết hợp giữa xử lý ảnh, trí tuệ nhân tạo, cơ sở dữ liệu, giao diện người dùng và điều khiển phần cứng. Hệ thống có thể nhận diện biển số ở cả cổng vào và cổng ra, kiểm tra dữ liệu trước khi mở barrier (thanh chắn/barie), đồng thời cập nhật trạng thái xe trong bãi sau khi xe ra khỏi cổng.

Kết quả thực nghiệm cho thấy phương pháp kết hợp YOLOv8 (mô hình phát hiện đối tượng phiên bản 8), PaddleOCR (bộ nhận dạng

ký tự quang học) và hậu xử lý theo quy tắc biến số Việt Nam đạt độ chính xác tốt trong điều kiện mô hình được căn chỉnh phù hợp. Hệ thống đáp ứng được mục tiêu của đề án và có thể tiếp tục mở rộng để tiêm cận một hệ thống bãi xe thông minh hoàn chỉnh hơn.